

On-prem setup runbook

APIConnect-catalog on Azure DevOps Server with a self-hosted Linux agent

v1.0 · 28 September 2026
companion to guide.pdf (sections 12–13)

This runbook takes you from the offline bundle to the first successful publish on an **on-prem Azure DevOps Server** (2020 or later; 2022 recommended) using **Azure Repos Git** and a **self-hosted Linux x64 agent** that has no internet access. Every step lists *where* to click and *what* to enter. Estimated time: 2–3 hours, most of it waiting for API keys and permissions.

What is different on Azure DevOps Server

Topic	On Azure DevOps Server
PR validation	The <code>pr:</code> block in <code>apic-ci.yml</code> is ignored for Azure Repos. PR builds are configured with a branch policy → Build validation on <code>main</code> (step 8).
Secrets	No Azure Key Vault. <code>APIC_APIKEY</code> is a secret variable (padlock) in the variable group; it is encrypted at rest and masked in logs.
Agents	No Microsoft-hosted pool. Set <code>AGENT_POOL</code> in <code>apic-ci.yml</code> to your self-hosted pool (step 6). Tools come from the bundle (<code>TOOLS_MODE=offline</code>).
Toolkit	Installed on the agent; <code>APIC_TOOLKIT_SOURCE=preinstalled</code> . The Azure Artifacts task is not even included in the compiled pipeline.
Environments / deployment jobs	Available since Server 2020. The five <code>apic-*</code> Environments are created automatically on the first run.
Certificates	Internal CAs are common on-prem: the agent must trust the Azure DevOps Server certificate <i>and</i> the three API Connect management ingresses (step 3).

You will need

- The bundle `APIConnect-catalog-bundle-<date>.zip` and its `.sha256`.
- A Linux x64 VM for the agent (2 vCPU / 4 GB is plenty): `bash 5`, `git ≥ 2.30`, `Python 3.9+` with `pip`, `unzip`, `sha256sum`, outbound HTTPS to the Azure DevOps Server and to the three API Connect management ingresses.
- From each API Connect instance's API Manager → *Tools for download*: the Linux CLI and `credentials.json` (3×).
- One IBM Cloud / toolkit API key per instance for a service identity that is a member of that instance's provider org with a publish role (3×).
- Azure DevOps Server: rights to create a project/repo, agent pool, variable groups, secure files, pipelines and branch policies (Project Administrator).

Part A — The agent machine

1 Install the bundled tools

Where: the agent VM, as a user with `sudo`

```
sha256sum -c APIConnect-catalog-bundle-*.zip.sha256          # integrity of the transfer
unzip APIConnect-catalog-bundle-*.zip && cd APIConnect-catalog-bundle-*
sudo bash tools/install-offline.sh                          # verifies SHA256SUMS, installs
yq/oasdiff/jq/spectral to /usr/local/bin, pip installs yamllint
```

If `pip install` is refused system-wide, the script falls back to `--user`; then `~/local/bin` of the *agent* user must be on `PATH` (see step 4).

2 Install the IBM API Connect toolkit

Where: API Manager of any instance (use the highest fix pack) → *Tools for download* → CLI (Linux); then the agent VM

```
sudo install -m 0755 apic-slim /usr/local/bin/apic          # or `apic` if you downloaded CLI
+ Designer
apic --accept-license version                                # must print the toolkit version,
10.0.8.x
```

3 Trust the internal certificates

Where: the agent VM. Skip if all endpoints use publicly trusted certificates.

```
# OS trust store (git, curl, most tools). Ubuntu/Debian:
sudo cp corp-root-ca.crt /usr/local/share/ca-certificates/ && sudo update-ca-certificates
# RHEL: sudo cp corp-root-ca.crt /etc/pki/ca-trust/source/anchors/ && sudo update-ca-trust
# Node-based tools (the Azure Pipelines agent tasks, and the apic toolkit - VERIFY for apic):
echo 'NODE_EXTRA_CA_CERTS=/etc/ssl/certs/ca-certificates.crt' | sudo tee -a /etc/environment
```

Test from the VM before going further: `curl -sS https://<ado-server>/tfs/ -o /dev/null -w '%{http_code}\n'` and `apic catalogs:list --server <ingress.management> --org <org>` after an interactive `apic iam-apikey login`. If `apic` reports a TLS error while `curl` works, the toolkit does not honour the OS store on your build; ask IBM support for the supported CA option of your toolkit version.

4

Register the Azure Pipelines agent

Where: Azure DevOps Server → *Collection settings* → *Agent pools* → *Add pool* (self-hosted, name `apic-linux`) → *New agent* → download the Linux x64 agent (it is served by your own server). Create a PAT with scope *Agent Pools* (*Read & manage*) for the registration.

```
sudo useradd -m -s /bin/bash azagent # dedicated user; never root
sudo mkdir -p /opt/azagent && sudo chown azagent /opt/azagent
sudo -iu azagent bash -c 'cd /opt/azagent && tar zxf ~/vsts-agent-linux-x64-*.tar.gz && \
    ./config.sh --unattended --url https://<ado-server>/tfs/DefaultCollection --auth pat --token
<PAT> \
    --pool apic-linux --agent apic-agent-01 --acceptTeeEula --replace'
# PATH for the service (systemd does not read the user's profile):
echo 'PATH=/usr/local/bin:/usr/bin:/bin:/home/azagent/.local/bin' | sudo -u azagent tee
/opt/azagent/.env
cd /opt/azagent && sudo ./svc.sh install azagent && sudo ./svc.sh start
# verify as the agent user: every line must print a path
sudo -iu azagent bash -c 'for t in yamllint spectral yq oasdiff jq apic git realpath; do
command -v $t || echo "MISSING $t"; done'
```

The pool must have *Grant access permission to all pipelines* (pool → Security), or you authorize the pipeline on its first run.

Part B — Repository and configuration

5 Import the repository into Azure Repos

Where: Azure DevOps Server → your project → *Repos* → *New repository* (Git, name `APIConnect-catalog`, **unchecked** "Add a README"); then any machine with git

```
git clone APIConnect-catalog.gitbundle APIConnect-catalog      # from the bundle, with history
cd APIConnect-catalog
git remote add origin https://<ado-server>/tfs/DefaultCollection/<project>/_git/APIConnect-
catalog
git push -u origin main
```

Then set `main` as the default branch (*Repos* → *Branches* → ... → *Set as default branch*) if it is not already.

6 Point the repository at your instances and your pool

Where: a branch + PR, or directly on `main` before the branch policy exists

File · setting	Value
<code>config/topology.yml</code> · <code>instances.dev.ingress.management</code>	The <code>apic --server</code> URL of the dev instance (platform-API / API Manager ingress), e.g. <code>https://api-manager.dev.apic.corp.local</code> . Same for <code>test</code> and <code>prod</code> .
<code>config/topology.yml</code> · <code>instances.*.ingress.gateway</code>	The gateway ingress of each instance (informational, used in docs/smoke tests).
<code>config/topology.yml</code> · <code>instances.*.org</code>	Provider organization name on each instance (<code>apic orgs:list --server ...</code> shows it).
<code>.azure-pipelines/apic-ci.yml</code> · <code>AGENT_POOL</code>	<code>apic-linux</code> (the pool from step 4)
<code>.azure-pipelines/apic-ci.yml</code> · <code>APIC_TOOLKIT_SOURCE</code>	<code>preinstalled</code> (already the default)

No `CHANGE ME` may remain in `topology.yml`: the pipeline warns on placeholders. Commit and push.

7

Library: variable groups and secure files

Where: *Pipelines* → *Library*

Object	Content
Variable group <code>apic-shared</code>	<code>TOOLS_MODE</code> = <code>offline</code> · <code>APIC_TOOLKIT_VERSION</code> = e.g. <code>10.0.8.9</code> (informational)
Variable group <code>apic-dev</code>	<code>APIC_APIKEY</code> = API key of the dev instance's publishing identity — click the padlock to make it secret
Variable group <code>apic-test</code>	<code>APIC_APIKEY</code> for the test instance (secret)
Variable group <code>apic-prod</code>	<code>APIC_APIKEY</code> for the prod instance (secret)
Secure files (tab <i>Secure files</i>)	Upload the three <code>credentials.json</code> as <code>apic-toolkit-credentials-dev.json</code> , <code>apic-toolkit-credentials-test.json</code> , <code>apic-toolkit-credentials-prod.json</code> (rename before upload — the file name is the identifier).

On each group and file: *Pipeline permissions* → either authorize the `APICConnect-catalog` pipeline after you created it (step 8), or "Open access" if your policy allows. Restrict who can *edit* `apic-prod` (group → Security).

Part C — Pipeline, policies, permissions

8

Create the pipeline and the PR policy

Where: *Pipelines* → *New pipeline* → *Azure Repos Git* → *APIConnect-catalog* → *Existing Azure Pipelines YAML file* → path `/ .azure-pipelines/apic-ci.yml` → **Save** (not Run). Rename it to `APIConnect-catalog` (... → Rename/move).

Then *Repos* → *Branches* → *main* → ... → *Branch policies*:

- **Require a minimum number of reviewers:** 1 (or more), "Allow requestors to approve their own changes" off.
- **Build validation** → +: pipeline `APIConnect-catalog`, trigger *Automatic*, policy requirement *Required*, build expiration *Immediately when main is updated*. This is what runs the four validation stages on every PR (the YAML `pr:` block is ignored on Azure Repos).
- Optional: **Automatically included reviewers** for `projects/*/prod/**` (the platform team) — a lightweight prod gate without pipeline approvals.

Effect of the policy

Nobody can push to `main` directly anymore — including you. Every change, including `topology.yml` edits, goes through a PR. That is intended.

9

Repository permissions for the build identity

Where: *Project settings* → *Repositories* → *APIConnect-catalog* → *Security*

Find **<project> Build Service (<collection>)** — or *Project Collection Build Service* if *Project settings* → *Pipelines* → *Settings* → "Limit job authorization scope to current project" is off — and set:

Permission	Value	Why
Read	Allow	checkout
Contribute	Allow	push the <code>published/<stage></code> tags
Create tag	Allow	same
Force push	Allow	the tags are moved with <code>git push --force refs/tags/...</code> ; branches are protected by the policy anyway

In *Project settings* → *Pipelines* → *Settings* keep "Protect access to repositories in YAML pipelines" on; the pipeline only needs its own repository.

First run and Environments

Where: a PR that touches only `projects/example-project/dev/` (e.g. edit a description)

1. Open the PR. The *Build validation* check starts the pipeline; the first time it asks to **permit** access to the variable groups, secure files and agent pool — click *Permit* for each. Expect the four stages to pass and the publish stages to be skipped (PR builds never publish).
2. Complete the PR. The merge run validates again and then runs `publish_dev` and `publish_nightly` in parallel — both report “No tag published/... yet: bootstrapping” and publish the whole `dev/` folder. `publish_integ`, `publish_rc` and `publish_prod` also bootstrap their folders; if you do not want that yet, temporarily remove those folders from the example project first.
3. *Pipelines* → *Environments* now shows `apic-dev`, `apic-nightly`, `apic-integ`, `apic-rc`, `apic-prod`. On each: *Approvals and checks* → + → **Exclusive lock**. On `apic-prod` optionally add **Approvals** with the release managers.
4. In each API Manager confirm the products show *Published*; call `GET /example/health` through each gateway ingress with a subscribed application.

Checklist and troubleshooting

	Done when...
☐	<code>sudo -iu azagent bash -c 'command -v yamllint spectral yq oasdiff jq apic git'</code> prints seven paths
☐	The agent shows <i>Online</i> in pool <code>apic-linux</code>
☐	<code>apic catalogs:list --server <ingress.management> --org <org></code> works from the agent VM for all three instances (after <code>apic iam-apikey</code>)
☐	<code>config/topology.yml</code> has no <code>CHANGE ME</code> ; <code>AGENT_POOL</code> is set in <code>apic-ci.yml</code>
☐	Variable groups <code>apic-shared</code> , <code>apic-dev</code> , <code>apic-test</code> , <code>apic-prod</code> exist; the API keys are padlocked
☐	Three secure files uploaded with the exact names <code>apic-toolkit-credentials-{dev,test,prod}.json</code>
☐	Branch policy on <code>main</code> : reviewers + build validation with the pipeline
☐	Build Service has Contribute, Create tag, Force push on the repository
☐	A PR turned green; the merge created <code>published/dev</code> and <code>published/nightly</code> tags (<i>Repos</i> → <i>Tags</i>)
☐	Exclusive lock on the five Environments

If something fails

Symptom	Fix
Pipeline never starts on a PR	Build validation policy missing or set to <i>Optional</i> /manual; the <code>pr:</code> YAML block does nothing on Azure Repos.
"No agent found in pool ... that satisfies the demands"	<code>AGENT_POOL</code> misspelt, agent offline, or the pipeline is not authorized on the pool (pool → Security → Pipeline permissions).
"offline mode: 'X' is not installed on this agent"	Tool missing for the <i>agent user</i> : rerun <code>install-offline.sh</code> , check <code>/opt/azagent/.env</code> PATH, restart the service (<code>sudo ./svc.sh restart</code>).
"the apic toolkit is not on PATH"	Step 2 not done on this agent, or installed under another user's home. Put it in <code>/usr/local/bin</code> .
Variable group / secure file "not authorized"	Open the run → <i>Permit</i> , or Library → object → Pipeline permissions → add the pipeline.
Login: TLS / "self signed certificate in certificate chain"	Step 3: CA into the OS store and <code>NODE_EXTRA_CA_CERTS</code> ; restart the agent service so it picks up <code>/etc/environment</code> .
Login: 401 / "user not found"	The API key's identity is not a member of that instance's provider org, or the key expired. Test interactively with the same <code>ingress.management</code> .

<code>git push ...</code> <code>refs/tags/published/...: TF401027 /</code> permission denied	Step 9 permissions for the right Build Service identity (project-scoped vs collection-scoped).
Stage <i>Skipped</i> although the folder changed	The change is not under <code>projects/<project>/<folder>/{apis,products}/</code> , or the tag already points at this commit (check <i>Repos</i> → <i>Tags</i>).
UniversalPackages / NodeTool task errors	They should not appear at all: confirm <code>APIC_TOOLKIT_SOURCE=preinstalled</code> and <code>TOOLS_MODE=offline</code> .

Day-2 reminders

Rotate the three API keys on a schedule (edit the variable, nothing in Git changes). After an API Connect fix-pack upgrade, reinstall the matching toolkit on the agent. To update the tools, rebuild the bundle on a connected machine and rerun `install-offline.sh` . Ingress changes are a one-line PR to `config/topology.yml` .